

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_0f586cf2-9eb\agent-  
a184b945aa512fc5c.jsonl`  
- SHA-256: `6aafef2e23bcf1bf9b3efa7593358d9d0ac455d0e3a2fa3e23bd5c85d0b27536`  
- Source modified: `2026-06-21T14:45:40+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_0f586cf2-9eb`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T14:42:18.601Z)

This is PR P0a from docs/PACKAGING_PLAN.md: introduce a resolved app-data HOME so a packaged app launched from an arbitrary CWD finds the same config/.env/DB/output, WITHOUT changing behaviour when RALLY_APP_HOME is unset. HARD CONTRACT: env unset => byte-identical to pre-P0a (CWD-relative). The repo: FastAPI engine, namespace package (no backend/__init__.py), hatchling editable install, windows-first users, requires-python>=3.10. Verified facts: full suite 1221 passed/7 skipped, ruff+mypy clean, coverage 85.10%, and an e2e proof showed config+DB land under RALLY_APP_HOME (nothing leaks to CWD) and == CWD when unset.

Changed/new files in the worktree C:/Users/avidu/Projects/bhi-wt-p0a:

```
- NEW backend/utils/app_home.py (the resolver:  
app_home/app_path/default_config_path/default_env_path/resolve_output_dir/default_os_app_home)  
- backend/config/loader.py (DEFAULT_CONFIG_PATH alias + use default_config_path())  
- backend/main.py (load_dotenv prefers app-home .env; static mount package-relative; output_dir  
via resolve_output_dir at server-start + trim)  
- scripts/doctor.py (writes config to default_config_path() not REPO_ROOT)  
- NEW tests/test_app_home.py
```

LENS: import-safety-ordering

Read backend/main.py top + backend/utils/app_home.py. ADVERSARIALLY check import-time safety: main.py:1-20 imports app_home BEFORE the heavy backend.config ? does importing backend.utils.app_home pull anything heavy or do filesystem I/O at import? Is load_dotenv ordering preserved (app-home .env if exists else walk-up)? The static mount does os.makedirs on a PACKAGE-relative dir ? on an INSTALLED wheel (site-packages, possibly read-only) does makedirs(exist_ok=True) raise? Is that a regression vs the old CWD makedirs? Flag any import-time crash or I/O risk.

Read the actual files under C:/Users/avidu/Projects/bhi-wt-p0a. Return findings with severity blocker|major|minor|nit, each with file:line and a concrete fix. Be harsh but precise; do not invent issues.

2. assistant (2026-06-21T14:42:21.345Z)

I'll review the import-safety and ordering concerns in this PR. Let me read the actual files.

3. user (2026-06-21T14:42:22.327Z)

```
1      """Resolved application-data HOME for a packaged, batteries-included install.  
2  
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,  
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and  
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it  
from  
6      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user  
silently
```

```

7      "loses" their index. This module introduces ONE resolver every state path roots on.
8
9      Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10     * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
sets it).
11     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13     The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/Library/Local/Share``)
14     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15     calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17     real user profile during a test run).
18
19     stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20     the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21     """
22
23     from __future__ import annotations
24
25     import os
26     import platform
27     from pathlib import Path
28
29     #: Environment variable the packaged launcher sets to pin the app-data home.
30     APP_HOME_ENV = "RALLY_APP_HOME"
31
32     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33     APP_DIR_NAME = "Khelsutra"
34
35
36     def app_home() -> Path:
37         """The resolved application-data home.
38
39         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41         """
42         env = os.environ.get(APP_HOME_ENV)
43         if env and env.strip():
44             return Path(env.strip()).expanduser()
45         return Path(".")
46
47
48     def app_path(*parts: str | os.PathLike[str]) -> Path:
49         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50         return app_home().joinpath(*[str(p) for p in parts])
51
52
53     def default_config_path() -> Path:
54         """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60

```

```

61     def default_env_path() -> Path:
62         """`<app-home>/env` ? the dotenv file the server prefers (e.g.
63         ``GEMINI_API_KEY``)."""
64         return app_home() / ".env"
65
66     def resolve_output_dir(output_dir: str) -> str:
67         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
68         through.
69         With ``RALLY_APP_HOME`` unset, ``resolve_output_dir("output") == "output"`` (CWD-
70         relative),
71         byte-identical to today; with it set, ``"output"`` becomes ``<app-home>/output`` so
72         the SQLite
73         DB and produced reels live alongside the rest of the app's state.
74         """
75         if not output_dir:
76             return str(app_home() / "output")
77         if os.path.isabs(output_dir):
78             return output_dir
79         return str(app_home() / output_dir)
80
81     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
82         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
83         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
84
85         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
86         * macOS ? ``~/Library/Application Support/<app_name>``
87         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/local/share/<app_name>`` if
88         unset)
89         """
90         system = platform.system()
91         if system == "Windows":
92             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
93         elif system == "Darwin":
94             base = str(Path.home() / "Library" / "Application Support")
95         else:
96             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
97         return Path(base) / app_name

```

4. user (2026-06-21T14:42:22.743Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    # PACKAGING_PLAN.md P0a: prefer an app-home .env (the packaged launcher pins
11    RALLY_APP_HOME),
12    # else fall back to the historical CWD+walk-up search so the repo/dev path is unchanged.
13    # stdlib-only import ? keeps this pre-pydanitic line light.
14    from backend.utils.app_home import default_env_path, resolve_output_dir
15
16    _env_path = default_env_path()
17    if _env_path.exists():
18        load_dotenv(_env_path)
19    else:
20        load_dotenv()

```

```

21     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
22     logging.basicConfig(
23         level=logging.INFO,
24         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
25         datefmt='%H:%M:%S'
26     )
27     logger = logging.getLogger(__name__)
28     from fastapi import FastAPI, HTTPException, Request
29     from fastapi.middleware.cors import CORSMiddleware
30     from pydantic import BaseModel
31     from typing import List, Dict, Any, Optional
32
33     from fastapi.staticfiles import StaticFiles
34     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
35     from backend.pipeline.validators.base import registry
36     from backend.infrastructure.database import Database
37     from backend.pipeline.segmenters.base import segmenter_registry
38     from backend.pipeline.stitcher.compiler import VideoCompiler
39     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
40     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
41     from backend.config import ( # new typed config
42         load_config as load_app_config,
43         AppConfig,
44         StitchingConfig,
45         enforce_startup_checks,
46         StartupCheckError,
47     )
48     from backend.results import Failure # standardized error/result contract
49     from backend.reporting import emit_indexer_report
50     from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
51     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
52     from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
53
54     app = FastAPI(title="Sports Highlight Indexer API")
55
56
57     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
58         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
59         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
60         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
61         src = os.environ if env is None else env
62         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
63         origins = [o.strip() for o in raw.split(",") if o.strip()]
64         return origins or ["*"]
65
66
67     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
68     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
69     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
70     # lock it to its origin; the middleware is fixed at startup.
71     app.add_middleware(
72         CORSMiddleware,
73         allow_origins=_cors_origins_from_env(),
74         allow_credentials=False,
75         allow_methods=["*"],

```

```

76         allow_headers=["*"],
77     )
78
79     # Serves static frontend files directly (now under api/ per approved structure).
80     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an

```

5. assistant (2026-06-21T14:42:25.523Z)

Let me read the rest of main.py around the static mount and output_dir handling.

6. user (2026-06-21T14:42:26.349Z)

```

79     # Serves static frontend files directly (now under api/ per approved structure).
80     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
81     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
82     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
83     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
84     os.makedirs(_STATIC_DIR, exist_ok=True)
85     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
86
87     @app.get("/", response_class=HTMLResponse)
88     def serve_index():
89         index_path = os.path.join(_STATIC_DIR, "index.html")
90         if os.path.exists(index_path):
91             with open(index_path, "r", encoding="utf-8") as f:
92                 return f.read()
93         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
94
95     # Global variables initialized at startup
96     db_instance: Optional[Database] = None
97     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
98
99     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
100     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
101     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
102     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
103     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
104     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
105     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
106     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
107         JobWaiting,
108         _arm_wake_timer,
109         _drain_due_jobs,
110         _get_executor,
111         _job_to_request,
112         _MAX_DRAIN_WAKE_SEC,
113         _maybe_record_job_cost,
114         _run_claimed_job,
115         _schedule_next_drain_if_waiting,
116     )
117
118     # Legacy thin wrapper for any remaining direct calls during transition.
119     # New code should import load_app_config / AppConfig from backend.config directly.
120     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
121         cfg = load_app_config(config_path)
122         return cfg.model_dump(mode="json")
123
124     # --- API Models ---

```

```

125     class ProcessRequest(BaseModel):
126         video_path: str
127         player_pool: Optional[List[str]] = None
128         max_frames: Optional[int] = None
129         segmenter: Optional[str] = None
130
131     class QueryRequest(BaseModel):
132         video_id: str
133         server: Optional[str] = None
134         receiver: Optional[str] = None
135         duration_min: Optional[float] = None
136         event_type: Optional[str] = None
137
138     class CompileRequest(BaseModel):

```

7. assistant (2026-06-21T14:42:29.323Z)

Now let me look at where `resolve\_output\_dir` is used at server-start and trim, and check the old behavior for the static mount. Let me find the relevant usages.

8. user (2026-06-21T14:42:35.448Z)

```

13:from backend.utils.app_home import default_env_path, resolve_output_dir
84:os.makedirs(_STATIC_DIR, exist_ok=True)
283:         # report.output_dir). Never raises. Surface the paths in the response.
407:     (which only writes into output_dir; the browser previously got back an unreachable
415:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
416:     path = os.path.join(output_dir, name)
418:     path = resolve_under_root(path, output_dir)
486:     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
487:     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
634:         out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
638:         os.makedirs(out_dir, exist_ok=True)
667:     # on the typed model (OutputConfig.output_dir), so every consumer ? including
670:     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
671:     global_config.output.output_dir = resolve_output_dir(args.output_dir)
672:     os.makedirs(global_config.output.output_dir, exist_ok=True)
674:     db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

```

9. user (2026-06-21T14:42:35.768Z)

```

7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching
(#289)
backend/main.py | 468 ++++++-----
1 file changed, 190 insertions(+), 278 deletions(-)

```

10. assistant (2026-06-21T14:42:39.155Z)

The diff stat shows 468 lines changed in main.py, which is far more than P0a should touch ? but that's likely because the worktree was cut from a different base. Let me focus on the actual P0a change. Let me check the original main.py on master for the static mount to compare against the regression claim.

11. user (2026-06-21T14:42:40.498Z)

```

26:from fastapi.staticfiles import StaticFiles
51:# Serves static frontend files directly (now under api/ per approved structure)

```

```

52:os.makedirs("backend/api/static", exist_ok=True)
53:app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
57:    index_path = "backend/api/static/index.html"
61:    return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
456:    os.makedirs(upload_dir, exist_ok=True)
748:        os.makedirs(out_dir, exist_ok=True)
779:    os.makedirs(global_config.output.output_dir, exist_ok=True)

```

12. assistant (2026-06-21T14:42:43.838Z)

Now I understand the static mount change clearly. Old: ``os.makedirs("backend/api/static", exist_ok=True)`` (CWD-relative). New: ``os.makedirs(_STATIC_DIR, exist_ok=True)`` where ``_STATIC_DIR`` is package-relative (inside site-packages on a wheel install).

Let me verify the loader.py change and confirm app_home import-time behavior, then check the doctor.py change.

13. user (2026-06-21T14:42:44.786Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Back-compat alias (was a module constant; no external consumers). Prefer
33     # ``default_config_path()`` ? it honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a).
With the
34     # env unset this is ``Path("config.json")`` (CWD-relative), exactly the pre-P0a value.
35     DEFAULT_CONFIG_PATH = default_config_path()
36
37
38     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
39         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
40         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
41         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
42         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""

```

```

43         out = dict(base)
44         for key, val in override.items():
45             if isinstance(val, dict) and isinstance(out.get(key), dict):
46                 out[key] = _deep_merge(out[key], val)
47             else:
48                 out[key] = val
49         return out
50
51
52     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
53         """The EXPLICITLY chosen deployment profile, env winning over config.json. ```` if
54         none
55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
56         a bare
57         environment never silently selects a profile (and never silently spends on cloud).
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
59         file's
60         profile (rather than suppressing it) ? so an env typo can't leave the recorded
61         profile
62         asserting a preset that was never applied. A bad value in config.json is left to
63         surface as
64         a hard, typed-Literal error downstream (config-file correctness)."""

```

14. user (2026-06-21T14:42:47.543Z)

```

diff --git a/backend/config/loader.py b/backend/config/loader.py
index 463d9ff..f937f23 100644
--- a/backend/config/loader.py
+++ b/backend/config/loader.py
@@ -10,16 +10,70 @@ from __future__ import annotations

import json
import logging
+import os
+from pathlib import Path
+from typing import Any

from pydantic import BaseModel

+from backend.utils.app_home import default_config_path
+
+from .models import AppConfig
+from .presets import (
+    COMPUTE_TARGET_FOR_PROFILE,
+    PROFILE_PRESETS,
+    is_known_profile,
+    preset_for,
+    suggest_profile,
+)

logger = logging.getLogger(__name__)

-DEFAULT_CONFIG_PATH = Path("config.json")
+
+## Back-compat alias (was a module constant; no external consumers). Prefer
+## ``default_config_path()`` ? it honours ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a). With the
+## env unset this is ``Path("config.json")`` (CWD-relative), exactly the pre-P0a value.
+DEFAULT_CONFIG_PATH = default_config_path()
+
+
+
+def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
+    """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge, scalars
+    replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
+    overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-profile
+    path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""

```

```

+ out = dict(base)
+ for key, val in override.items():
+     if isinstance(val, dict) and isinstance(out.get(key), dict):
+         out[key] = _deep_merge(out[key], val)
+     else:
+         out[key] = val
+ return out
+
+
+def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
+    """The EXPLICITLY chosen deployment profile, env winning over config.json. ```` if none
+    is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so a bare
+    environment never silently selects a profile (and never silently spends on cloud).
+
+    An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the file's
+    profile (rather than suppressing it) ? so an env typo can't leave the recorded profile
+    asserting a preset that was never applied. A bad value in config.json is left to surface as
+    a hard, typed-Literal error downstream (config-file correctness)."""
+    env_profile = os.environ.get("DEPLOYMENT_PROFILE")
+    if env_profile and env_profile.strip():
+        ep = env_profile.strip()
+        if is_known_profile(ep):
+            return ep
+        logger.warning(
+            "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring it "
+            "and falling back to config.json deployment.profile.", ep, ",
+    ".join(PROFILE_PRESETS),
+    )
+    # fall through to the file's profile (env typo must not suppress a valid file choice)
+    dep = file_data.get("deployment")
+    if isinstance(dep, dict):
+        prof = dep.get("profile")
+        if isinstance(prof, str) and prof.strip():
+            return prof.strip()
+    return ""

def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str = "") ->
list[str]:
@@ -60,7 +114,7 @@ def load_config(path: str | Path | None = None) -> AppConfig:
    Missing file or unreadable file ? returns a fully defaulted AppConfig.
    Unknown keys in the file are tolerated (extra="allow" on the model).
    """
-    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
+    cfg_path = Path(path) if path is not None else default_config_path()

    file_data: dict[str, Any] = {}
    if cfg_path.exists():
@@ -72,6 +126,15 @@ def load_config(path: str | Path | None = None) -> AppConfig:
    # In production we might log here.
    logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")

+    # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `x`, `null`) would
+    # otherwise crash startup: a truthy non-mapping hits `.items()` in _unknown_key_paths, and
+    any
+    # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults + warn,
+    per the
+    # loader's "bad input is non-fatal" contract.
+    if not isinstance(file_data, dict):
+        logger.warning("%s is not a JSON object (got %s); ignoring it and using defaults.",
+            cfg_path, type(file_data).__name__)
+        file_data = {}
+
    if file_data:
        unknown = _unknown_key_paths(file_data, AppConfig)

```

```

        if unknown:
@@ -82,8 +145,55 @@ def load_config(path: str | Path | None = None) -> AppConfig:
            cfg_path, "", ".join(sorted(unknown)),
        )

- # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
- merged = {**_get_builtin_defaults(), **file_data}
+ # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied at
+ # consumption sites downstream). A *deployment profile* is opt-in ? it is applied ONLY when
+ # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile). With no
+ # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
+ defaults = _get_builtin_defaults()
+ profile = _resolve_explicit_profile(file_data)
+
+ if profile and not is_known_profile(profile):
+     # Only an unrecognised value from config.json reaches here (env typos already fell back
+     # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits the
+     # typed Literal at model_validate below, a hard clear error ? loud, never silent.
+     logger.warning(
+         "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no preset.",
+         profile, "", ".join(PROFILE_PRESETS),
+     )
+     profile = ""
+
+ if profile:
+     merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
+     # Record the profile actually applied ? the env may have chosen it over config.json,
+     # so re-stamp it onto the merged deployment section to keep the record honest.
+     merged.setdefault("deployment", {})
+     if isinstance(merged["deployment"], dict):
+         merged["deployment"]["profile"] = profile
+         ct = merged["deployment"].get("compute_target")
+         canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
+         if not ct and canonical:
+             # An active profile with no explicit compute_target (e.g. config.json blanked
it
+             # to "") gets its canonical target back ? the record never silently lies about
an
+             # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
+             merged["deployment"]["compute_target"] = canonical
+         elif ct and canonical and ct != canonical:
+             logger.warning(
+                 "[CONFIG] deployment profile %r normally uses compute_target=%r but "
+                 "config.json overrides it to %r ? honoring the explicit override.",
+                 profile, canonical, ct,
+             )
+             logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
+                 profile, merged["deployment"].get("compute_target"))
+     else:
+         # No profile ? the original shallow merge (Pydantic re-defaults nested sections).
+         merged = {**defaults, **file_data}
+         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only (D15)
+         if suggestion:
+             logger.debug(
+                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests %r ? "
+                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect never "
+                 "auto-applies ? D15).", suggestion,
+             )

        return AppConfig.model_validate(merged)

@@ -94,7 +204,7 @@ def save_default_config(path: str | Path | None = None) -> Path:
    Used by scripts/doctor.py diagnostics and the --setup CLI path.

```

```

    Overwrites if the file exists.
    """
-   cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
+   cfg_path = Path(path) if path is not None else default_config_path()
    cfg = AppConfig()
    cfg_path.parent.mkdir(parents=True, exist_ok=True)
    cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
diff --git a/scripts/doctor.py b/scripts/doctor.py
index 5c30f61..309431b 100644
--- a/scripts/doctor.py
+++ b/scripts/doctor.py
@@ -22,9 +22,13 @@ def run_pip_install():
    requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
    pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r", requirements_path]

+   import platform
+   if has_gpu:
+       print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
+       pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
+   elif platform.system() == "Darwin":
+       # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
+       print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
+   else:
+       print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
+       pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
@@ -45,12 +49,16 @@ def init_default_config():
    """
    try:
        from backend.config import save_default_config
+       from backend.utils.app_home import default_config_path
    except Exception as e:
        print(f"[ERROR] Could not import backend.config loader: {e}")
        print("[INFO] Falling back to no-op for config initialization.")
        return

-   config_path = os.path.join(REPO_ROOT, "config.json")
+   # PACKAGING_PLAN.md P0a: write to the SAME app-home config path the server READS
+   # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root while
+   # the server read from the CWD, so --setup and the server disagreed once CWD ? repo root.
+   config_path = str(default_config_path())
    if os.path.exists(config_path):
        print(f"[INFO] {config_path} already exists. Skipping initialization.")
        return
@@ -91,13 +99,17 @@ def run_diagnostics():
    print("[FAIL] FFprobe is NOT found in your PATH.")

    if not (ffmpeg_ok and ffprobe_ok):
+       import platform
+       print("\n" + "!" * 50)
+       print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
+       print("How to install FFmpeg on Windows:")
+       print("  Option 1: Open PowerShell as Administrator and run:")
+       print("      winget install Gyan.FFmpeg")
+       print("  Option 2: Download the build from https://ffmpeg.org/download.html")
+       print("      and add its 'bin' directory to your System PATH environment
variable.")
+       _sys = platform.system()
+       if _sys == "Windows":
+           print("Install on Windows:  winget install Gyan.FFmpeg")
+       elif _sys == "Darwin":
+           print("Install on macOS:    brew install ffmpeg")
+       else:
+           print("Install on Linux:    sudo apt-get install -y ffmpeg  (or your distro's
package manager)")

```

```
+         print("Or download a build from https://ffmpeg.org/download.html and put ffmpeg/ffprobe
on PATH.")
        print("!" * 50 + "\n")
    else:
        print("[SUCCESS] All essential media command line tools are ready.")
```

15. assistant (2026-06-21T14:42:52.511Z)

The diff confirms the worktree was cut from a base that also includes the M1 profile-preset loader work, so loader.py has more changes than strictly P0a ? but those are unrelated and out of scope for my lens. Let me focus on the import-safety/ordering concerns.

Now the critical question: the ``DEFAULT_CONFIG_PATH = default_config_path()`` is evaluated at `**import time**` of ``backend.config.loader``. This snapshots ``RALLY_APP_HOME`` at import. But ``load_config`` correctly calls ``default_config_path()`` fresh each time, so it's not used for the live path. Let me verify ``DEFAULT_CONFIG_PATH`` has no remaining consumers.

16. user (2026-06-21T14:42:53.737Z)

```
C:\Users\avidu\Projects\bhi-wt-p0a\docs\CODE_MAP.md:132: - `::DEFAULT_CONFIG_PATH =
Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_built_in_defaults` =
`AppConfig().model_dump(...)`
C:\Users\avidu\Projects\bhi-wt-p0a\backend\config\loader.py:35:DEFAULT_CONFIG_PATH =
default_config_path()
```

17. assistant (2026-06-21T14:42:58.757Z)

``DEFAULT_CONFIG_PATH`` has no code consumers (only the loader defines it and a doc references it). It's import-time-frozen and dead ? a latent bug if anyone uses it later, since it would ignore a launcher that sets ``RALLY_APP_HOME`` after import. Minor.

Now let me examine the central concern of the lens: the ``os.makedirs(_STATIC_DIR, exist_ok=True)`` at import time on a read-only wheel install. Let me check if there's a try/except guard and confirm the import ordering of ``load_dotenv``. Let me also verify the test file covers these.

18. user (2026-06-21T14:42:59.102Z)

```
1     """Tests for the resolved application-data HOME (PACKAGING_PLAN.md P0a).
2
3     The load-bearing guarantee: with ``RALLY_APP_HOME`` UNSET, every resolver is byte-
identical to the
4     pre-P0a CWD-relative behaviour; with it SET, config/.env/output/DB all root under that
one home.
5     """
6
7     import os
8     from pathlib import Path
9
10    import pytest
11
12    from backend.utils import app_home as ah
13    from backend.config import loader
14
15
16    @pytest.fixture(autouse=True)
17    def _clear_app_home_env(monkeypatch):
18        """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
19        monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
20
21
22    # ---- default (env unset) is byte-identical to today ----
23
24    def test_app_home_defaults_to_cwd():
```

```

25         assert ah.app_home() == Path(".")
26
27
28     def test_default_config_path_unset_is_cwd_config_json():
29         # Pre-P0a this was the module constant Path("config.json").
30         assert ah.default_config_path() == Path("config.json")
31
32
33     def test_default_env_path_unset_is_cwd_dotenv():
34         assert ah.default_env_path() == Path(".env")
35
36
37     def test_resolve_output_dir_unset_is_identity_for_relative():
38         assert ah.resolve_output_dir("output") == "output"
39         assert ah.resolve_output_dir("custom") == "custom"
40
41
42     # ---- RALLY_APP_HOME set ? everything roots there ----
43
44     def test_app_home_honors_env(monkeypatch, tmp_path):
45         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
46         assert ah.app_home() == tmp_path
47
48
49     def test_app_home_expands_user(monkeypatch):
50         monkeypatch.setenv(ah.APP_HOME_ENV, "~/khelsutra-data")
51         assert ah.app_home() == Path.home() / "khelsutra-data"
52
53
54     def test_app_home_ignores_blank_env(monkeypatch):
55         monkeypatch.setenv(ah.APP_HOME_ENV, " ")
56         assert ah.app_home() == Path(".")
57
58
59     def test_config_and_env_paths_root_under_home(monkeypatch, tmp_path):
60         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
61         assert ah.default_config_path() == tmp_path / "config.json"
62         assert ah.default_env_path() == tmp_path / ".env"
63         assert ah.app_path("output", "x.db") == tmp_path / "output" / "x.db"
64
65
66     def test_resolve_output_dir_set(monkeypatch, tmp_path):
67         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
68         assert ah.resolve_output_dir("output") == str(tmp_path / "output")
69
70
71     def test_resolve_output_dir_absolute_passthrough(monkeypatch, tmp_path):
72         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
73         abs_dir = str(tmp_path / "elsewhere") if os.path.isabs(str(tmp_path)) else
os.path.abspath("elsewhere")
74         assert ah.resolve_output_dir(abs_dir) == abs_dir
75
76
77     def test_resolve_output_dir_empty_falls_back_to_home_output(monkeypatch, tmp_path):
78         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
79         assert ah.resolve_output_dir("") == str(tmp_path / "output")
80
81
82     # ---- default_os_app_home: the OS-conventional dir the launcher exports (NOT auto-
selected) ----
83
84     def test_default_os_app_home_windows(monkeypatch, tmp_path):
85         monkeypatch.setattr(ah.platform, "system", lambda: "Windows")
86         monkeypatch.setenv("APPDATA", str(tmp_path / "Roaming"))
87         assert ah.default_os_app_home() == tmp_path / "Roaming" / "Khelsutra"

```

```

88
89
90     def test_default_os_app_home_macos(monkeypatch):
91         monkeypatch.setattr(ah.platform, "system", lambda: "Darwin")
92         assert ah.default_os_app_home() == Path.home() / "Library" / "Application Support" /
"Khelsutra"
93
94
95     def test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
96         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
97         monkeypatch.setenv("XDG_DATA_HOME", str(tmp_path / "share"))
98         assert ah.default_os_app_home() == tmp_path / "share" / "Khelsutra"
99
100
101     def test_default_os_app_home_linux_default(monkeypatch):
102         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
103         monkeypatch.delenv("XDG_DATA_HOME", raising=False)
104         assert ah.default_os_app_home() == Path.home() / ".local" / "share" / "Khelsutra"
105
106
107     # ---- loader integration: load_config() roots on the app-home ----
108
109     def test_loader_reads_config_from_app_home(monkeypatch, tmp_path):
110         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
111         (tmp_path / "config.json").write_text('{"sport": "tennis"}', encoding="utf-8")
112         cfg = loader.load_config() # no explicit path ? default_config_path() under the
home
113         assert cfg.sport == "tennis"
114
115
116     def test_loader_no_config_in_home_uses_defaults(monkeypatch, tmp_path):
117         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path)) # empty dir, no config.json
118         cfg = loader.load_config()
119         # Defaults load cleanly (no crash, no file) ? same contract as a missing CWD
config.json.
120         assert cfg is not None
121         assert hasattr(cfg, "indexing")
122
123
124     def test_save_default_config_writes_under_app_home(monkeypatch, tmp_path):
125         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
126         written = loader.save_default_config() # no explicit path
127         assert written == tmp_path / "config.json"
128         assert written.exists()
129
130
131     def test_explicit_path_still_overrides_app_home(monkeypatch, tmp_path):
132         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
133         explicit = tmp_path / "explicit.json"
134         explicit.write_text('{"sport": "pickleball"}', encoding="utf-8")
135         cfg = loader.load_config(str(explicit))
136         assert cfg.sport == "pickleball"
137

```

19. assistant (2026-06-21T14:43:03.866Z)

Now let me verify the actual import-time behavior of `app\_home.py` and the makedirs-on-wheel concern empirically. The key adversarial test: does `os.makedirs(\_STATIC\_DIR, exist\_ok=True)` raise on a read-only directory? `exist\_ok=True` only suppresses `FileExistsError` ? it does NOT suppress `PermissionError`. Since the directory `backend/api/static` is shipped in the wheel (it contains index.html), it will already exist, so makedirs returns immediately without attempting creation. Let me verify the static dir is actually shipped (committed) so it always exists post-install.

20. user (2026-06-21T14:43:05.762Z)

```
total 48
drwxr-xr-x 1 avidu 197609      0 Jun 21 19:59 .
drwxr-xr-x 1 avidu 197609      0 Jun 21 20:04 ..
-rw-r--r-- 1 avidu 197609 16350 Jun 21 19:59 app.js
-rw-r--r-- 1 avidu 197609  8743 Jun 21 19:59 index.html
-rw-r--r-- 1 avidu 197609 13418 Jun 21 19:59 style.css
---tracked---
backend/api/static/app.js
backend/api/static/index.html
backend/api/static/style.css
```

21. assistant (2026-06-21T14:43:12.635Z)

Important nuance. The static dir is tracked and shipped, so on a normal wheel install it exists and ``makedirs(exist_ok=True)`` is a no-op (returns early without touching the filesystem). But there's a subtlety with ``exist_ok=True`` and intermediate path components ? ``os.makedirs`` with ``exist_ok=True`` does NOT raise if the leaf exists. Let me confirm the exact CPython semantics: when the target already exists as a directory and ``exist_ok=True``, `makedirs` calls ``mkdir`` which raises ``FileExistsError``, then checks ``exist_ok`` and `path.isdir()`` and swallows it. It does not attempt to create parents that already exist beyond the recursion. So no `PermissionError` on an existing tree.

The real regression risk would only materialize if the static dir were NOT shipped (empty dir not tracked by git ? not in wheel). Let me check whether hatchling includes the static data files in the wheel, and whether wheel packaging would include these non-.py files.

22. user (2026-06-21T14:43:12.902Z)

```
1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__.d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
```

```

37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47         "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
48     ]
49
50     # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
51     security.edge_auth_enabled=True.
52     # `jwt` is lazy-imported, so the base runtime works without this; install it for
53     hosted/multi-tenant.
54     auth = ["PyJWT[crypto]>=2.8"]
55
56     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
57     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
58     # fetched over SSH; if you lack access, simply omit this group ? reporting
59     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
60     # pip install -e ../sports-obsreport --no-build-isolation
61     reporting = [
62         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
63     ]
64
65     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
66     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
67     # express, so they are NOT declarable here. Install them out-of-band instead:
68     # # CUDA 12.4 box:
69     # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
70     # https://download.pytorch.org/whl/cu124
71     # # CPU-only box (also what CI uses):
72     # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
73     gpu = []
74
75     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
76     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
77     # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
78     R2 +
79     # DO Spaces + MinIO via endpoint_url.) `gdrive` needs NO extra ? it reads a locally
80     # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
81     backend/storage/gdrive.py.
82     storage-s3 = ["boto3>=1.34"]
83     storage-gcs = ["google-cloud-storage>=2.16"]
84
85     [project.scripts]
86     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
87     indexer = "backend.main:main"
88
89     [tool.hatch.metadata]
90     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
91     # reference*); hatchling requires opting in to allow those in metadata.
92     allow-direct-references = true
93
94     [tool.hatch.build.targets.wheel]
95     # `backend` is a namespace package (no __init__.py); list both import roots
96     # explicitly so an editable/wheel install exposes `import backend` and
97     # `import training` regardless of cwd.
98     packages = ["backend", "training"]
99

```

This is a real, important finding. Hatchling's wheel target with ``packages = ["backend", "training"]`` includes the ``backend`` package. By default hatchling includes `**all files**` under the listed packages (not just ``.py``), so ``backend/api/static/*.{html,js,css}`` would normally be packaged. Let me verify by actually building the wheel and inspecting whether the static assets are included ? this determines whether the makedirs regression is real or theoretical.

24. user (2026-06-21T14:43:19.315Z)

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import hatchling; print('hatchling', hatchling.__version__)
    ^^^^^^^^^^^^^^^^^^^^^
ModuleNotFoundError: No module named 'hatchling'
no build module
```

25. assistant (2026-06-21T14:43:22.233Z)

Let me try building the wheel in an isolated way to inspect its contents, which is the most reliable check for whether static assets ship.

26. user (2026-06-21T14:43:34.282Z)

```
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - hatchling
* Getting build dependencies for wheel...
* Building wheel...
Successfully built badminton_highlight_indexer-0.1.0-py3-none-any.whl
```

27. assistant (2026-06-21T14:43:36.411Z)

Wheel built. Now let me inspect whether the static assets are inside it.

28. user (2026-06-21T14:43:38.970Z)

```
STATIC files in wheel:
  backend/api/static/app.js
  backend/api/static/index.html
  backend/api/static/style.css
total entries: 145
any backend/api/ entries:
  backend/api/__init__.py
  backend/api/auth.py
  backend/api/job_worker.py
  backend/api/uploads.py
  backend/api/static/app.js
  backend/api/static/index.html
  backend/api/static/style.css
```

29. assistant (2026-06-21T14:43:48.929Z)

Confirmed: the static assets DO ship in the wheel. So on a normal installed wheel, ``backend/api/static/`` exists (it contains `index.html/app.js/style.css`), and ``os.makedirs(_STATIC_DIR, exist_ok=True)`` is a no-op early-return ? it does NOT attempt to write into site-packages, so no ``PermissionError``.

The makedirs-on-wheel concern is therefore `**NOT a regression in practice**` for the shipped product. However, it's still slightly more fragile than the old code in one narrow edge: the old code created ``.backend/api/static`` under CWD (always writable, user-owned); the new code targets a path inside the install location. If a packager ever strips the static assets (e.g. ships an empty static via a different bundling path in P1a, or zips without empty-dir entries) AND site-packages is read-only (system Python, root-installed), the import-time ``os.makedirs``

would raise `PermissionError` and crash the whole server at import. Since `exist\_ok=True` does not catch `PermissionError`, this is an unguarded import-time write into the install tree. Worth a defensive guard given P1a will rework static bundling.

Let me verify the import-time safety of app_home.py itself and confirm load_dotenv ordering precisely. Let me also double check that importing app_home does no I/O.

30. user (2026-06-21T14:43:52.654Z)

```
FS-write calls during import of app_home: []
module deps loaded: []
app_home() == WindowsPath('.')
```

31. assistant (2026-06-21T14:44:02.950Z)

Confirmed: importing `backend.utils.app\_home` does zero filesystem writes and pulls nothing heavy (no torch/cv2/pydantic/numpy). The import-ordering claim in the module docstring holds ? it's stdlib-only and safe to import before `backend.config`.

Now let me verify the `load\_dotenv` ordering precisely. The concern: pre-P0a behavior was `load\_dotenv()` (walk-up from CWD). New behavior: if app-home `.env` exists, load that; ELSE `load\_dotenv()` (walk-up). When `RALLY\_APP\_HOME` is unset, `default\_env\_path()` is `Path(".env")`. So if a `.env` exists in CWD, the new code calls `load\_dotenv(Path(".env"))` instead of `load\_dotenv()`. Is that byte-identical?

This is a subtle behavioral difference. `load\_dotenv()` with no args walks UP from the CWD to find `.env` (via `find\_dotenv`). `load\_dotenv(Path(".env"))` loads exactly `./env` and does NOT walk up. Let me check: when env is unset and a `./env` exists, both load it. But when env is unset and NO `./env` exists but a `.env` exists in a PARENT dir, the old code (`load\_dotenv()`) would find it via walk-up, while the new code: `Path(".env").exists()` is False ? falls to `load\_dotenv()` ? walks up ? finds parent `.env`. So the else branch preserves walk-up.

The only divergence: env unset, `./env` EXISTS. Old: `load\_dotenv()` ? `find\_dotenv()` finds `./env` (usually) ? loads it. New: `load\_dotenv(Path(".env"))` ? loads `./env`. Same file. Let me check whether `load\_dotenv()` default `override` differs and whether find_dotenv would ever pick a different file than `./env` when `./env` exists.

32. user (2026-06-21T14:44:05.629Z)

```
load_dotenv sig: (dotenv_path: Union[str, ForwardRef('os.PathLike[str]'), NoneType] = None,
stream: Optional[IO[str]] = None, verbose: bool = False, override: bool = False, interpolate:
bool = True, encoding: Optional[str] = 'utf-8') -> bool
find_dotenv sig: (filename: str = '.env', raise_error_if_not_found: bool = False, usecwd: bool =
False) -> str
find_dotenv uses frame/usecwd: True | walks parents: False
```

33. assistant (2026-06-21T14:44:08.697Z)

This is a subtle but important finding for the HARD CONTRACT. `find\_dotenv` with the default `usecwd=False` walks up starting from the **caller's frame directory** (the directory of `backend/main.py`), NOT the CWD. Let me confirm by reading the find_dotenv source.

34. user (2026-06-21T14:44:09.911Z)

```
def find_dotenv(
    filename: str = ".env",
    raise_error_if_not_found: bool = False,
    usecwd: bool = False,
) -> str:
    """
    Search in increasingly higher folders for the given file

    Returns path to the file if found, or an empty string otherwise
    """
```

```

def _is_interactive():
    """Decide whether this is running in a REPL or IPython notebook"""
    if hasattr(sys, "ps1") or hasattr(sys, "ps2"):
        return True
    try:
        main = __import__("__main__", None, None, fromlist=["__file__"])
    except ModuleNotFoundError:
        return False
    return not hasattr(main, "__file__")

def _is_debugger():
    return sys.gettrace() is not None

if usecwd or _is_interactive() or _is_debugger() or getattr(sys, "frozen", False):
    # Should work without __file__, e.g. in REPL or IPython notebook.
    path = os.getcwd()
else:
    # will work for .py files
    frame = sys._getframe()
    current_file = __file__

    while frame.f_code.co_filename == current_file or not os.path.exists(
        frame.f_code.co_filename
    ):
        assert frame.f_back is not None
        frame = frame.f_back
    frame_filename = frame.f_code.co_filename
    path = os.path.dirname(os.path.abspath(frame_filename))

    for dirname in _walk_to_root(path):
        check_path = os.path.join(dirname, filename)
        if _is_file_or_fifo(check_path):
            return check_path

    if raise_error_if_not_found:
        raise IOError("File not found")

    return ""

```

35. assistant (2026-06-21T14:44:22.408Z)

This confirms a meaningful nuance, but let me reason carefully about whether it's actually a behavioral change.

Pre-P0a, the old `backend/main.py` called bare `load_dotenv()` ? `find_dotenv(usecwd=False)`. That walks up from the `caller` frame's directory == `dirname(backend/main.py)` = `<repo>/backend/`, then walks up to `<repo>/` etc. So the OLD code, when launched from the repo root, would find `<repo>/env` via the frame walk-up ? NOT necessarily a CWD `env`.

New P0a code (env unset): `default_env_path()` = `Path("env")` (CWD-relative). `_env_path.exists()` checks `./env` relative to `CWD`.
 - If launched from repo root, CWD == repo root, so `./env` == `<repo>/env` ? exists ? `load_dotenv(Path("env"))`. Same file the old frame-walk found. Identical.
 - But if launched from a SUBDIRECTORY of the repo (CWD = `<repo>/scripts`, say) with no `./env` there: new code `Path("env").exists()` is False ? falls to `load_dotenv()` ? frame walk-up from `backend/` ? finds `<repo>/env`. Same as old.

So the else-branch preserves the old frame-walk-up. The only case where they differ: `CWD` has a `env` that the old frame-walk-up would NOT have reached (e.g., launched from an unrelated directory containing a `env`). Old code: frame-walk from `backend/` finds repo `env` (or none). New code: loads the CWD `env` first. This is a genuine divergence, but only in an exotic launch-from-foreign-dir-with-dotenv scenario. Given the stated contract is about CWD-relative state, the new behavior (prefer CWD `env`) is arguably more correct, and within the repo/dev/CI path (launched from repo root) it is byte-identical. I'll note this as a minor/nit precision point on the "byte-identical" claim rather than a blocker, since the standard dev/CI/test

invocation is unaffected.

Let me verify the server-start path (lines 630-675) to confirm output_dir resolution and DB path, and that `args.output\_dir` defaults preserve behavior.

36. user (2026-06-21T14:44:23.075Z)

```
620         return
621
622     if args.trim_start is not None and args.trim_end is not None:
623         if not args.video_path:
624             print("[ERROR] Please provide --video-path to extract a segment.")
625             return
626
627         # Determine output path
628         out_filename = args.trim_output or "trimmed_segment.mp4"
629         if os.path.isabs(out_filename):
630             out_path = out_filename
631             out_dir = os.path.dirname(out_path)
632             out_name = os.path.basename(out_path)
633         else:
634             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
635             out_path = os.path.join(out_dir, out_filename)
636             out_name = out_filename
637
638         os.makedirs(out_dir, exist_ok=True)
639         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
640
641         # global_config isn't initialized this early; load the typed config so the trim
642         # clip carries the same configured stitching paddings + watermark as a real
643         # highlight (watermark default-on).
644         stitching = load_app_config().stitching
645         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
646                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
647         try:
648             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
649             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
650         except Exception as e:
651             print(f"[ERROR] Failed to compile trimmed segment: {e}")
652         return
653
654     if args.debug_clip is not None:
655         if not args.video_path:
656             print("[ERROR] Please provide --video-path to debug a specific clip.")
657             return
658         run_cli_diagnose_clip(args.video_path, args.debug_clip)
659         return
660
661     # Initialize Global Config and DB
662     global global_config, db_instance
663     global_config = load_app_config() # returns typed AppConfig
664     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
665
666     # The CLI --output-dir overrides the configured output directory. It now lives
667     # on the typed model (OutputConfig.output_dir), so every consumer ? including
668     # /api/compile ? reads the same value instead of a write-only runtime hack.
669     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
670     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
671     global_config.output.output_dir = resolve_output_dir(args.output_dir)
```

```

672         os.makedirs(global_config.output.output_dir, exist_ok=True)
673
674         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
675         db_instance = Database(db_path)
676
677         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
678         # the executor is in-process. Mark them failed so pollers see a terminal state.
679         swept = db_instance.fail_stale_jobs()

```

37. assistant (2026-06-21T14:44:27.002Z)

This confirms the server-start path correctly uses ``resolve_output_dir(args.output_dir)`` at runtime (not import time), and DB path derives from it. When env unset, ``resolve_output_dir("output") == "output"`` ? byte-identical. Good.

Let me check what ``args.output_dir`` defaults to, to ensure the resolve call gets a relative default (not None/absolute) preserving identity.

38. user (2026-06-21T14:44:28.622Z)

```

13:from backend.utils.app_home import default_env_path, resolve_output_dir
283:         # report.output_dir). Never raises. Surface the paths in the response.
407:         (which only writes into output_dir; the browser previously got back an unreachable
415:         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
416:         path = os.path.join(output_dir, name)
418:         path = resolve_under_root(path, output_dir)
486:         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
487:         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
593:         parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
604:         parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")
634:         out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
666:         # The CLI --output-dir overrides the configured output directory. It now lives
667:         # on the typed model (OutputConfig.output_dir), so every consumer ? including
669:         # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
670:         # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
671:         global_config.output.output_dir = resolve_output_dir(args.output_dir)
672:         os.makedirs(global_config.output.output_dir, exist_ok=True)
674:         db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

```

39. assistant (2026-06-21T14:44:33.465Z)

``--output-dir`` defaults to ``"output"`` (relative), so ``resolve_output_dir("output")`` returns ``"output"`` when env unset ? byte-identical. Good.

One more adversarial check on the ``resolve_output_dir`` HARD-CONTRACT claim: when env is unset, ``app_home()`` returns ``Path(".")``, so ``str(Path(".") / "output")``. On Windows that's ``"output"``? Let me verify ? ``Path(".") / "output"`` could stringify as ``"output"`` or ``".\\output"``. This matters for byte-identity of the DB path and config-recorded ``output_dir``.

40. user (2026-06-21T14:44:36.599Z)

```

Path(.) / output -> 'output'
Path(.) / config.json -> 'config.json'
default_config_path unset == 'config.json'
resolve_output_dir(output) == 'output'
default_config_path() == 'config.json'
default_env_path() == '.env'

```

41. assistant (2026-06-21T14:44:43.786Z)

Byte-identity confirmed on Windows: ``resolve_output_dir("output") == "output"`,`
``default_config_path()`` stringifies to ``"config.json"`,` ``default_env_path()`` to ``".env"`. The
`Path(".") / x` normalizes away the `.` prefix. The HARD CONTRACT holds for these paths.`

Note: ``resolve_output_dir("output")`` takes the relative branch (``str(app_home() / output_dir)``),
and the empty-string branch (``str(app_home() / "output")``) also yields ``"output"`. Both byte-
identical.`

Now let me confirm the one remaining import-ordering subtlety: ``backend/main.py`` line 13 imports
``app_home`` BEFORE pydantic-heavy ``backend.config`` (line 41). I verified `app_home` is stdlib-only.
But line 13's import of ``resolve_output_dir`` also implicitly imports the ``backend.utils`` package
``__init__``. Let me check that ``backend/utils/__init__.py`` doesn't pull anything heavy, since
that runs before ``backend.config`` too.

42. user (2026-06-21T14:44:45.539Z)

EXISTS:

1 backend/utils/__init__.py

Utils package

43. assistant (2026-06-21T14:44:50.222Z)

``backend/utils/__init__.py`` is a one-line comment ? no heavy imports. The import-ordering claim
fully holds: line 13 pulls only stdlib via ``app_home``, keeping the pre-pydantic ``load_dotenv``
path light.

I have thoroughly examined all the lens concerns. Let me do a final check on the exact CPython
makedirs semantics to be precise in my finding about whether the existing-dir case could ever
raise, and whether there is a try/except guard around line 84.

44. user (2026-06-21T14:44:56.046Z)

existing-dir mkdirs(exist_ok=True): OK (no raise)
missing-leaf under RO parent: no raise (Windows perms lenient)

45. assistant (2026-06-21T14:45:06.043Z)

Confirmed: ``os.makedirs(existing, exist_ok=True)`` is a clean no-op when the dir exists (the
shipped-wheel case), and ``exist_ok=True`` does NOT swallow ``PermissionError`` for a missing leaf
(Windows is lenient with chmod, but POSIX root-installed site-packages would raise).

I have all the evidence I need. Summary of my adversarial import-safety/ordering review:

1. ****app_home.py import safety****: VERIFIED clean ? stdlib-only (os, platform, pathlib), zero FS
writes at import, pulls nothing heavy. The docstring claim that it's safe to import before
``backend.config`` holds. ``backend/utils/__init__.py`` is empty.
2. ****load_dotenv ordering****: Mostly preserved, with one narrow divergence from "byte-identical"
? the old bare ``load_dotenv()`` does a frame-relative walk-up (from ``backend/``), while the new
code prefers a CWD ``.`.env`` first. Identical for the standard repo-root/dev/CI launch; differs
only when launched from a foreign directory that happens to contain a ``.`.env``.
3. ****static-mount mkdirs on a wheel****: Confirmed the static assets DO ship in the wheel (built
and inspected it), so the dir always exists post-install and ``mkdirs(exist_ok=True)`` is a
clean no-op ? NOT a regression for the shipped product. But it's an unguarded import-time write
into the install tree that would raise ``PermissionError`` (not swallowed by ``exist_ok``) if a
future packaging step (P1a reworks static bundling) ships without the dir on read-only POSIX
site-packages, crashing the server at import.
4. ****DEFAULT_CONFIG_PATH**** is import-time-frozen and has no code consumers ? dead/latent
footgun.

5. ****Byte-identity of resolved paths**** when env unset: VERIFIED on Windows.

46. user (2026-06-21T14:45:34.182Z)

Structured output provided successfully

47. assistant (2026-06-21T14:45:40.867Z)

I've completed the adversarial import-safety/ordering review of PR P0a. All findings have been returned via StructuredOutput.

Key conclusions:

- ****No blockers/majors.**** The import-time safety claim holds ? I empirically verified ``backend.utils.app_home`` does zero FS writes at import and pulls nothing heavy, so it's genuinely safe to import before the pydantic-backed ``backend.config``.
- The static-mount ``os.makedirs`` is NOT a regression today (I built the wheel and confirmed the static assets ship, so the dir always exists ? `makedirs` is a no-op), but it's an unguarded import-time write into the install tree that could crash on read-only POSIX site-packages once P1a reworks bundling. Filed as minor with a guard fix.
- ``load_dotenv`` ordering is preserved for the normal repo-root/dev/CI launch but has a narrow non-byte-identical edge (CWD ``env`` now wins over the historical frame-relative walk-up). Minor.
- ``DEFAULT_CONFIG_PATH`` is import-time-frozen with no code consumers ? a latent footgun. Nit.